

Graph of Thoughts: Solving Elaborate Problems with Large Language Models

Maciej Besta, Nils Blach, Ales Kubice , Robert Gerstenberger, Michał Podstawski, Lukas Gianinazzi
Joanna Gajda, Tomasz Lehmann Hubert Niewiadomski , Piotr Nyczyk , Torsten Hoefler
preprint, 2024

2024. 03. 17

김준철, 권성은, 이관호, 조수빈, 조해창, 황지현

Contents

- Background
- Introduction
- The GoT Framework
- System Architecture & Extensibility
- Example Use Cases
- The Latency-Volume Tradeoff
- Evaluation
- Conclusion

Background IO (Input-Output)

IO의 특징

- LLM 추론의 가장 일반적인 입출력 형태
- Input sequence x 를 바탕으로 output y 를 바로 예측

“예시없이 정답을 예측하는 Zero-Shot이나 N 개의 예시를 바탕으로 예측하는 Few-Shot으로 예측”

Zero Shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

x

➡ A: The answer is 11. y

One Shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

x_1

A: The answer is 11.

y_1

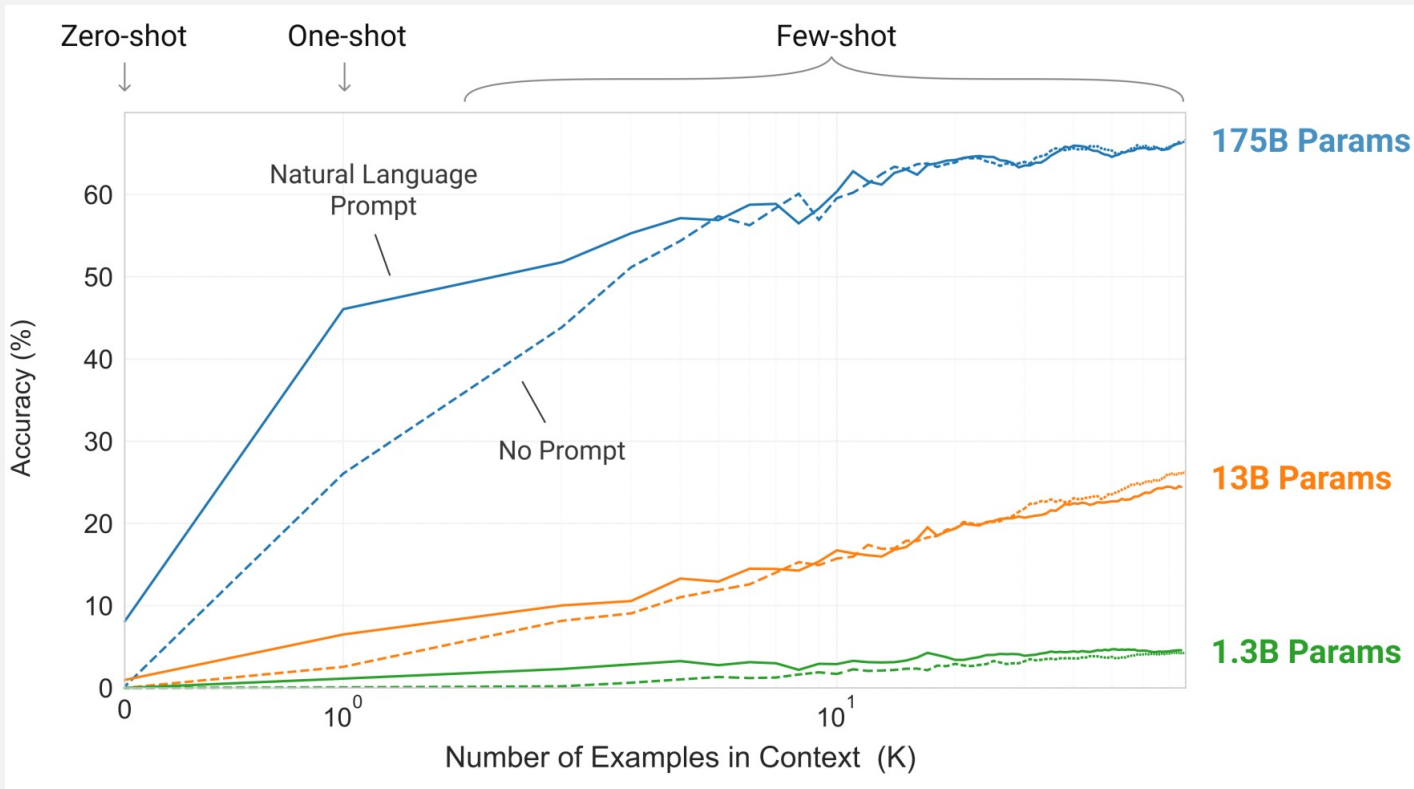
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

x_2

➡ A: The answer is 9. y_2

Background IO (Input-Output)

Fine-Tuning 없이 성능 올리는 방안



1. Natural Language Prompt 작성
- 문제 해결에 적합한 지시사항 작성
2. Number of Examples in Context
- 문제 해결에 유의미한 여러 예시를 작성

➡ Natural Language Prompt, Few-Shot 외에 LLM을 튜닝하지 않고 성능을 개선하는 연구가 진행되기 시작

Background Chain-of-Thought (CoT)

Chain-of-Thought의 특징

- Input sequence x 와 정답 y 사이에 논리적인 “생각” a 를 추가하는 형태
- 생각(Thought) : 결과를 도출하기까지 생성되는 토큰단위의 근거나 해답

Standard One-Shot Prompt

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought One-Shot Prompt

Model Input

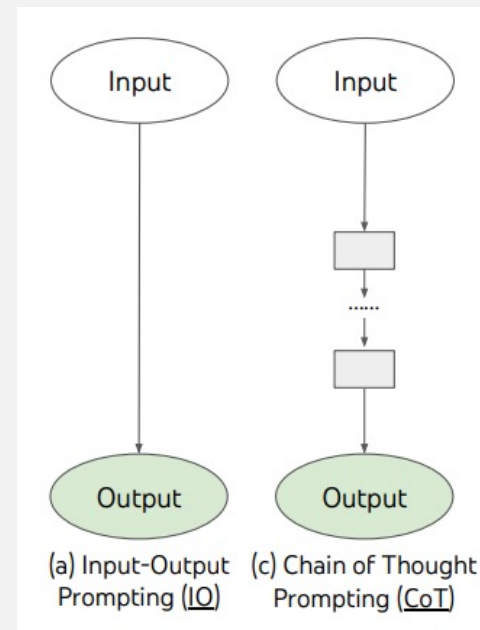
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Background Chain-of-Thoughts (CoT)

Chain-of-Thought 의 장단점

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

- Chain-of-Thought prompting의 장점

1. Allows models to **decompose multi-step** problems into intermediate steps
2. Provides an **interpretable window** into the behavior of the model
3. Can be used for tasks such as math word problems, commonsense reasoning and symbolic manipulation. And CoT is **potentially applicable to any task**
4. Can be readily elicited in sufficiently large off-the-shelf language models simply by **including examples of chain of thought sequences into the exemplars** of few-shot prompting

- Chain of Thoughts prompting의 개선할 점

1. CoT has only one reasoning path
2. No guarantee of correct reasoning paths

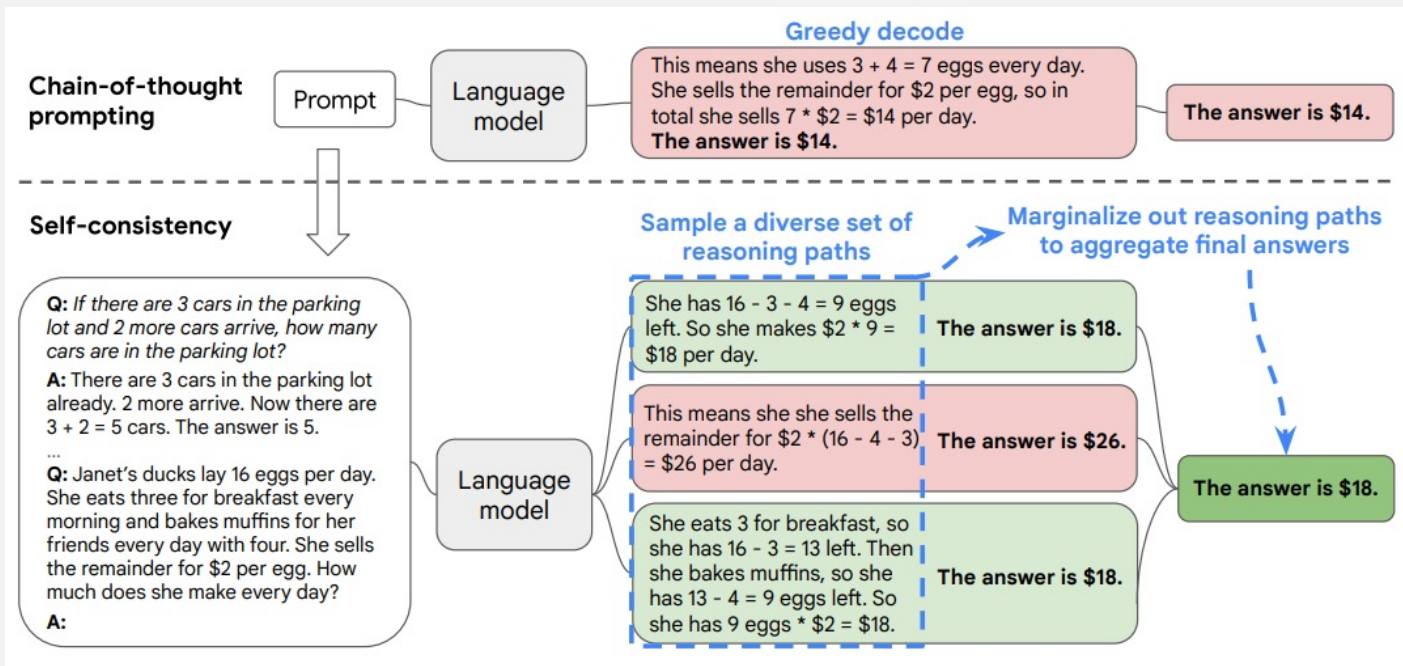


간단한 형태의 Reasoning의 추가만으로 문제해결능력 향상되었으나 하나의 갈래의 Reasoning만 가능한 문제를 개선해야함

Background Self-Consistency with CoT (CoT-SC)

Self-Consistency with CoT 의 특징 및 장단점

- 여러 방식으로 CoT를 생성하도록 한 이후에 가장 빈도가 높은 예측값을 채택함 (temperature sampling, top-k sampling, nucleus sampling)
- 여러 개의 reasoning path로부터 가장 일관성 있는 답변을 채택

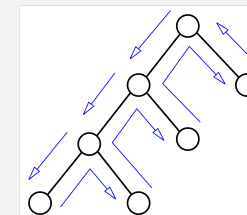


- CoT-SC 의 장점

1. Can explore different reasoning path

- CoT-SC 의 개선할 점

1. Does not offer local exploration
ex) backtracking



- Backtracking

잘못된 reasoning이 발견되었을 때
검증하여 이전으로 되돌아가는 기능

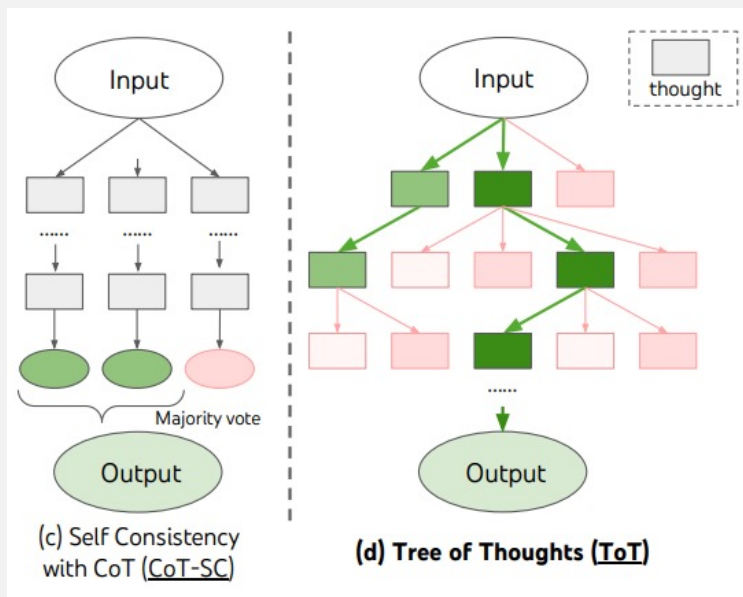
2. Left-to-right, Token level 예측만 가능함



다수의 CoT로 더 좋은 성능을 기대할 수 있게 되었지만 Reasoning path를 결론 도출 전에 검증할 수 있도록 개선해야함

Background Tree of Thoughts (ToT)

Tree of Thought 의 특징



- Thought Decomposition

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

- CoT (Implicit Thought Decomposition)

Input → Extract number and calculate → Output

- ToT (Explicit Thought Decomposition)

Input → Extract number → Evaluate → Output

- State evaluator

노드마다 가장 좋은 선택

- Value Prompt

“score 1~10”, “select good, soso, bad”

1점 3점 5점

결과마다 프롬프트 실행을 잘했는지 독립적으로 평가

- Vote Prompt

“what is the best choice in this examples?”

동일한 단계의 결과 내에서 가장 적합한 결과 채택

- Thought Generation

하나의 노드에서 여러 브랜치 생성

- Sample

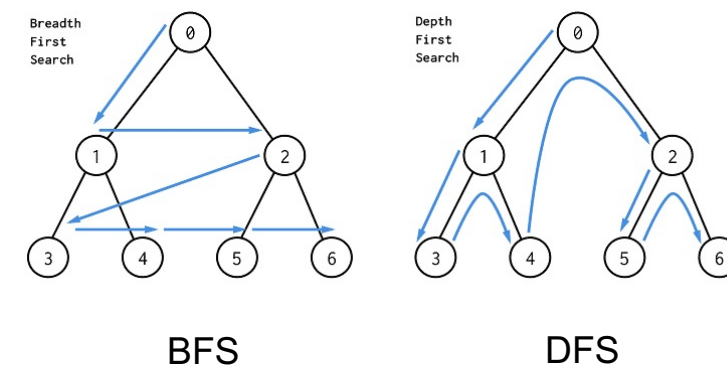
CoT 방식으로 생성

- Propose Prompt

“Generate possible next steps”

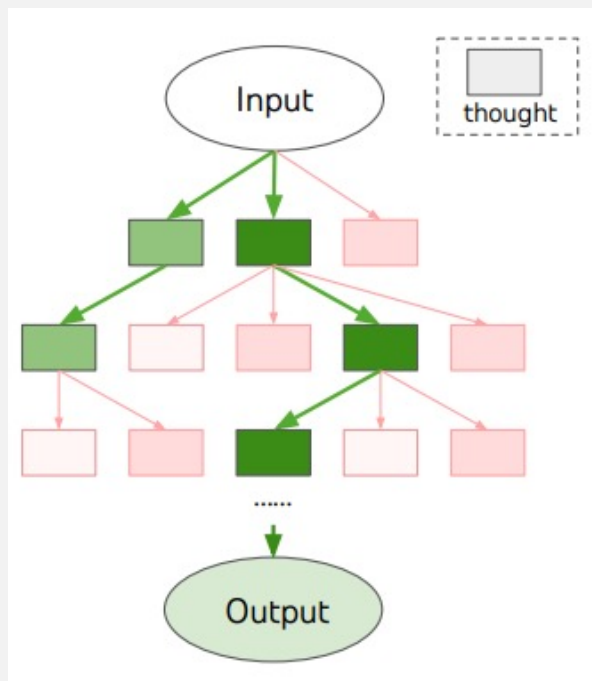
다음 단계에서 적합한 행동을 하도록 유도

- Search Algorithm



Background Tree of Thoughts (ToT)

Tree of Thoughts 의 장단점



Tree of Thoughts 의 장점

1. IO, CoT, CoT-SC 방식이 모두 가능함
2. 전체 과정을 generation, evaluation, search algorithm으로 분리해서 관리 가능
3. 각 노드별로 다른 LLM이나 리소스 운용이 가능함
4. 여러 갈래의 생각마다 평가를 진행하여 완성도 높은 Reasoning path를 유지하기 용이함

Tree of Thoughts 의 개선할 점

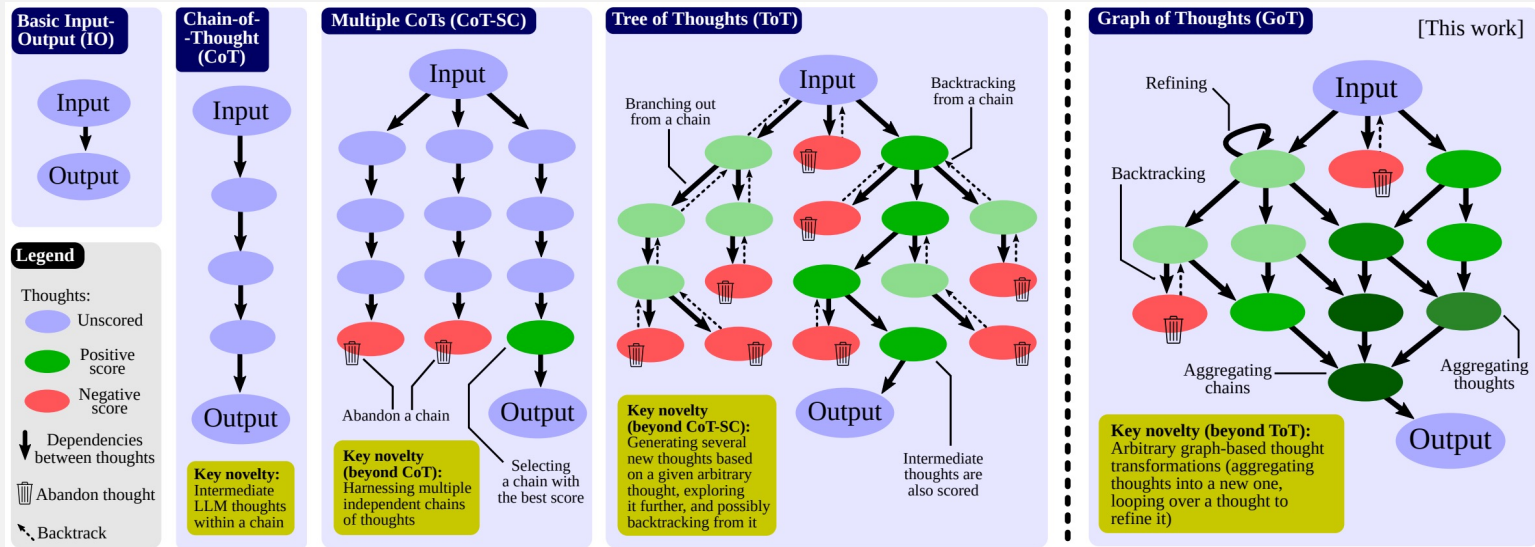
1. 사고가 진행되는 Reasoning path의 구조가 너무 단조롭고 유연하지 않음
2. 만들어지는 Thoughts의 대부분이 낭비됨

➡ 완성도 높은 reasoning path를 만들 수 있으나 생각을 활용하는 구조가 단순하고 대부분이 낭비되는 단점을 개선해야함

Introduction

Comparison with other prompting strategies

- Architecture Comparison



- Qualitative Comparison

Scheme	Sc?	Mc?	Tr?	Ag?
Chain-of-Thought (CoT) [71]	🗑️	✖	✖	✖
Self-Consistency with CoT [67]	🗑️	🗑️	✖	✖
Thought decomposition [75]	🗑️	🗑️	🗑️	✖
Tree-of-Thought (ToT) [43]	🗑️	🗑️	🗑️	✖
Tree of Thoughts (ToT) [77]	🗑️	🗑️	🗑️	✖
Graph of Thoughts (GoT)	🗑️	🗑️	🗑️	🗑️

Table 1: Comparison of prompting schemes, with respect to the supported transformations of thoughts. “Sc?”: single chain of thoughts? “Mc?”: multiple chains of thoughts? “Tr?”: tree of thoughts? “Ag?”: arbitrary graph of thoughts? “🗑️”: full support, “🗑️”: partial support, “✖”: no support.

- GoT는 더 유연해진 구조를 통해 각 노드의 활용방법 확대
- GoT는 이전의 다양한 프롬프트 방법론을 모두 포괄할 수 있는 그래프 구조

- 프롬프트 방법론마다 구현가능한 형태 시각화

다만 본 논문에서 제시한 GoT 는 자연스럽게 subtask로 분리할 수 있는 task에 적합하며
 분리된 subtask 각각 수행하고 최종적으로 통합하는 형태로 구현됨
 (아키텍처의 구조가 Merge Sort와 유사하다고 볼 수 있음)

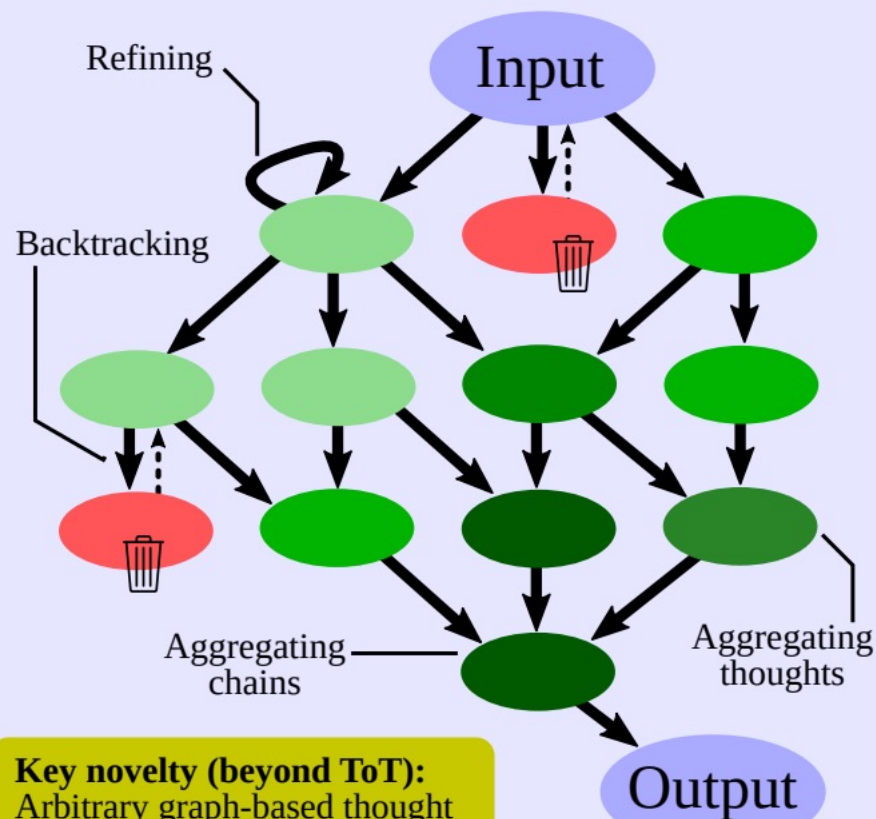
Introduction

Contributions

1. Networked reasoning 을 활용하여 LLM의 활용 범위 확장
기존의 prompting 방식으로 다루지 못하던 문제를 해결
2. Maximize accuracy and minimize cost
정확성만 높이는데 치중하지 않고 비용까지 고려
3. Graph based paradigm
그래프를 활용한 새로운 방식의 프롬프팅 방법론 제시
4. SOTA performance
특정 Task에서 가장 좋은 성능 달성
5. Volume of thought : new evaluating a prompting strategy
효과적인 프롬프팅을 평가할 수 있는 새로운 metric 제시

Graph of Thoughts (GoT)

[This work]



Key novelty (beyond ToT):
Arbitrary graph-based thought transformations (aggregating thoughts into a new one, looping over a thought to refine it)

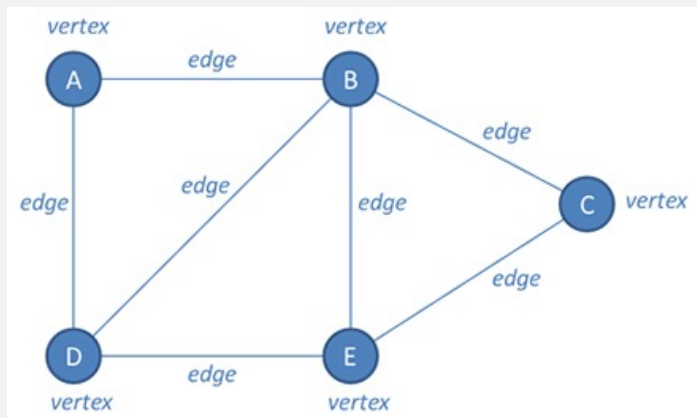
The GoT Framework

- 01** Reasoning Process
- 02** Transformations of Thoughts
- 03** Scoring & Ranking Thoughts

GoT Framework Reasoning Process

GoT Reasoning Process Architecture

- General Graph

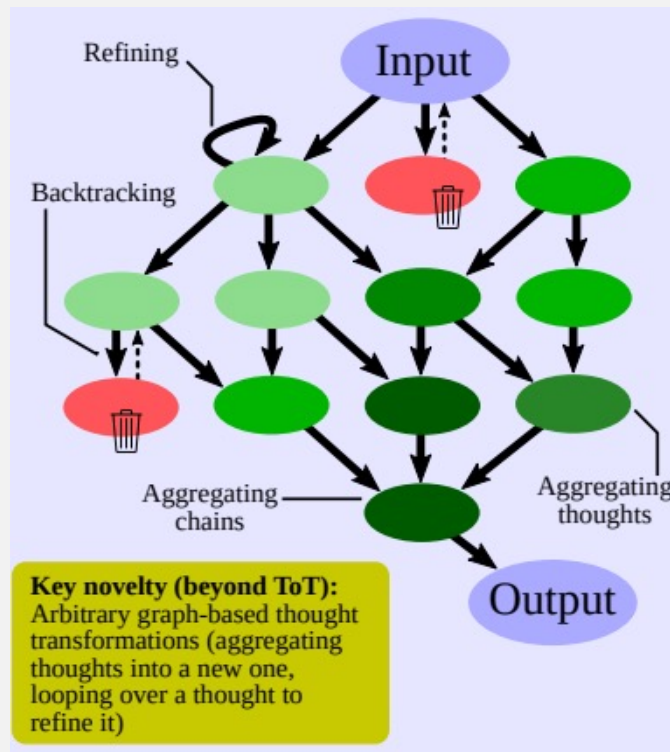


$$G = (V, E)$$

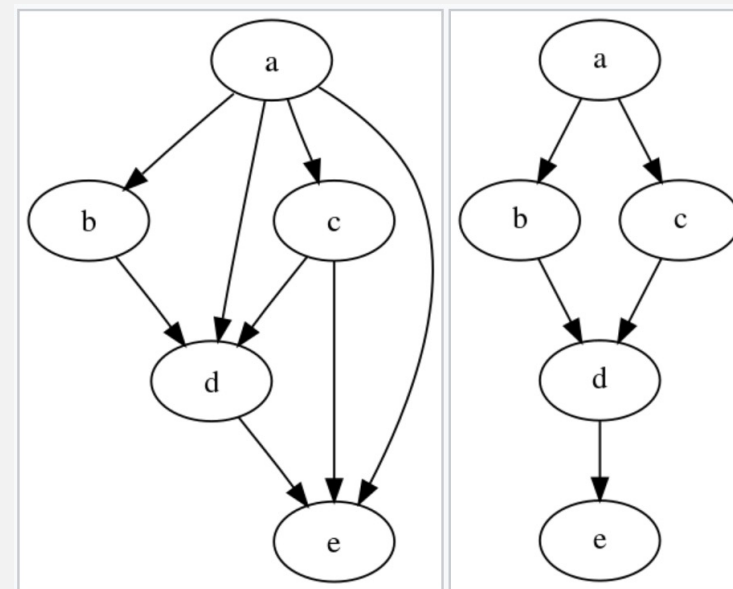
$$V = \{A, B, C, D, E\}$$

$$E = \{AA, AB, AC, AD, AE, \\ BA, BB, BC, BD, BE, \\ CA, CB, CC, CD, CE, \\ DA, DB, DC, DD, DE, \\ EA, EB, EC, ED, EE\}$$

- GoT Graph



- Directed Acyclic Graph

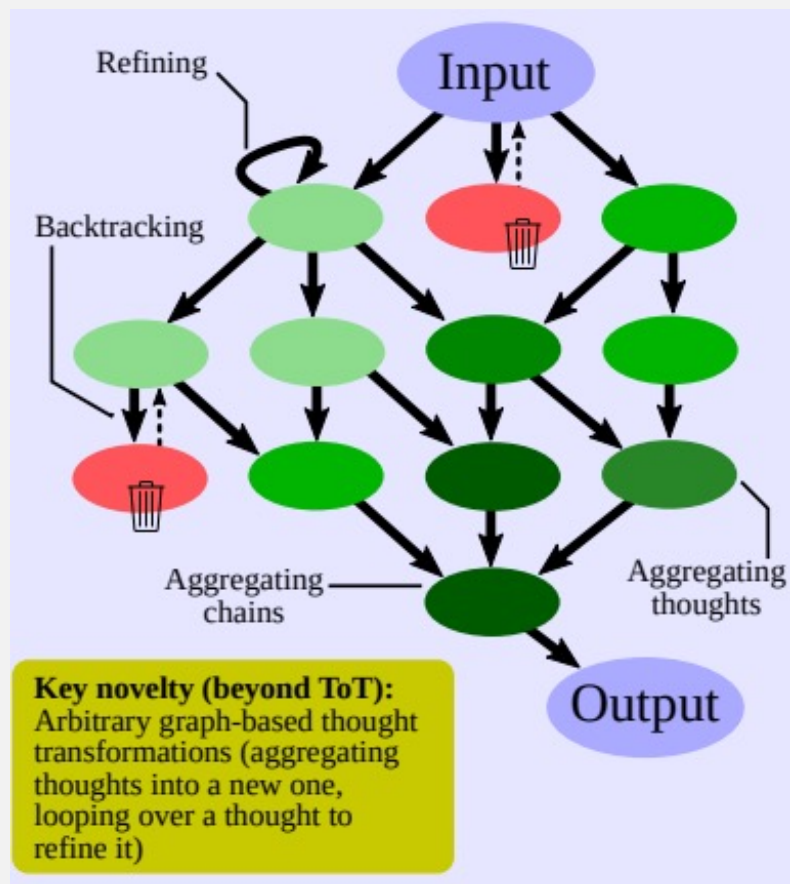


We model the reasoning process as a directed graph $G = (V, E)$

➡ 모든 그래프 구조를 포괄하는 것이 아닌 단방향의 순환하지 않는 그래프로 범위를 정함

GoT Framework Reasoning Process

Association Graph with the LLM reasoning process



- Heterogeneous Graph

$$G = (V, E, c)$$

1. 각 그래프가 Vertex, Edge 외에 각 Vertex를 class로 매핑하는 c 로 정의 가능
2. 노드별로 각각 다른 역할, 다른 프롬프트로 Reasoning을 진행함

- Thoughts Transformation

1. 그래프의 특성을 LLM의 Reasoning과 연결하기 위해 Thoughts Transformation을 적용
2. 좋은 평가를 받은 LLM 결과를 모두 활용하여 하나로 병합하거나 하나의 노드 내에서 반복적으로 강화하는 형태로 구현 가능
3. 이 방법론은 CoT, CoT-SC, ToT 의 변형된 형태로 확장이 가능함

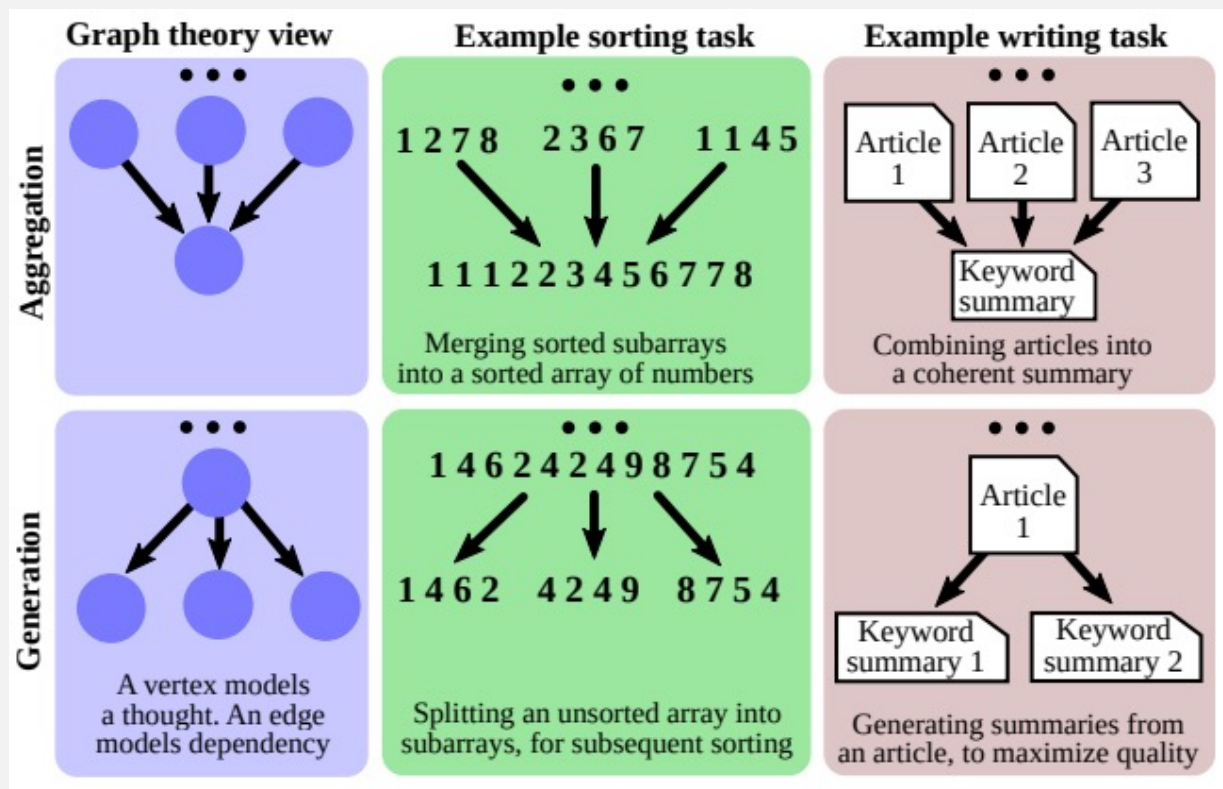
GoT Framework Transformations of Thoughts

Association Graph with the LLM reasoning process

- graph-enabled transformations

LLM의 Reasoning 그래프 형태로 구현하면서 얻는 이점

LLM의 프롬프팅으로 인해 그래프가 확장되면서 유용한 것들만 병합하여 활용하고 유용하지 않은 것들은 삭제하는 방식으로 아키텍처 구성이 가능함



- Refining Transformations
 - $V^+ = \{\}$: 추가적으로 생성되는 node는 없음
 - $E^+ = \{(v, v)\}$: 자신으로 향하는 edge만 추가
- Aggregation Transformations
 - 이전에 존재했던 v_1 부터 v_k 까지의 LLM결과를 활용하여 병합
 - 그래프 내의 여러 추론결과 중 유용한 노드를 활용하여 병합
- Generation Transformations
 - 하나의 v 에서 더 좋은 결과를 도출하기 위해 v_1^+ 부터 v_k^+ k개로 추론
 - ToT처럼 가능한 경우의 수만 늘리는 과정이라기 보다 이후 병합되는데 유용한 지식을 추출하는 것으로 해석 할 수 있음

GoT Framework **Scoring & Ranking Thoughts**

Definition of Scoring and Ranking

Scoring

$$\mathcal{E}(v, G, p_{\theta}):$$

$\mathcal{E}$: scoring function

v : vertex (LLM thought)

G : Graph

p_{θ} : LLM

- 하나의 노드 v 를 평가
- 일반화를 위해 이전까지의 모든 그래프를 활용하여 평가
- 경우에 따라 LLM를 활용하여 평가

Ranking

$$\mathcal{R}(G, p_{\theta}, h)$$

$\mathcal{R}$: Ranking function

G : Graph

p_{θ} : LLM

h : number of highest-ranking thoughts

- 그래프 전체에서 랭킹을 계산할 수 있는 모든 노드가 대상
- 가장 점수가 높은 h 개의 노드 선택
- 경우에 따라 LLM을 활용하여 랭킹

Scoring과 Ranking은 각 Task 별로 적합한 전략을 채택

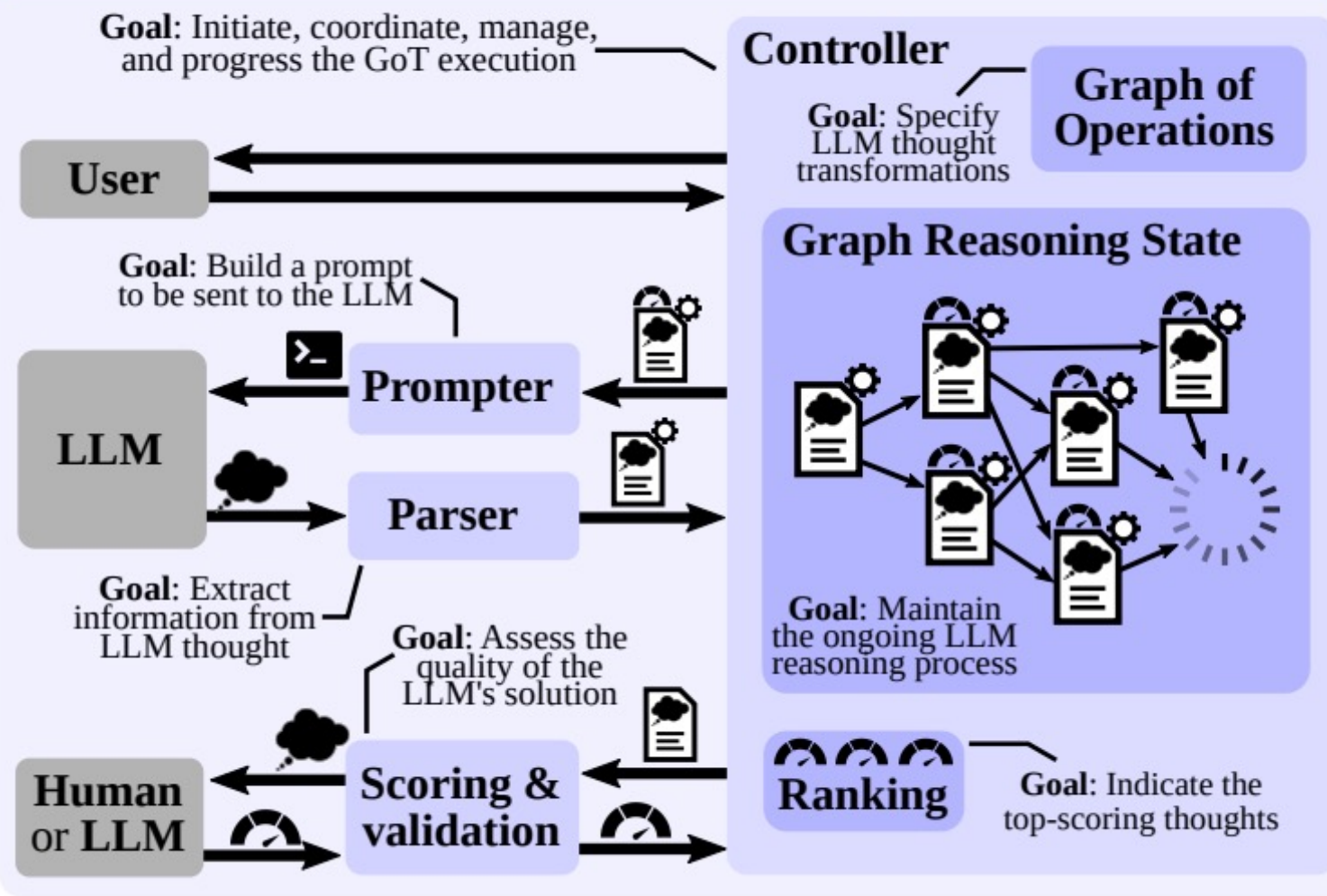
System Architecture & Extensibility

01 System Architecture

02 Extensibility

System Architecture & Extensibility **System Architecture**

Architecture overview



Controller

- 모든 Reasoning 과정을 통제하는 모듈
 1. Graph of Operations
 - 어떤 방식으로 문제해결전략을 진행할지 정의
 2. Graph Reasoning State
 - 전체 Reasoning 과정을 state로 관리
 3. Ranking
 - 가장 높은 점수의 LLM 결과 선택

Prompter

- 각 단계에 LLM에게 지시사항을 작성

Parser

- LLM의 결과를 관리가능한 형태로 추출

Scoring & Validation

- LLM의 결과에 대해 평가

System Architecture & Extensibility

Extensibility

API for Controller

- ➔ //LLM params: model used, temperature, max tokens, api key, org, ...
- ➔ //LLM cost features: prompt token cost, response token cost, ...
- ➔ //Instances of Prompter + Parser + Graph of Operations,
- ➔ //Any additional input parameters (e.g., numbers to be sorted).

Available operations when building the GoO (extensible)

- ➔ Generate, Aggregate, Score, ... //see Prompter API
- ➔ KeepBest(N) //preserves N best scoring thoughts
- ➔ Repeat(k) //Repeat a given operation k times, generating k thoughts.
//For example, this enables "Aggregate" to generate multiple outcomes
//of the combination operation. Each such thought is maintained
//within the Graph Reasoning State and scored individually.

API for Prompter (extensible)

- ➔ Generate(t, k) //generate a prompt for k new thoughts, using thought t
- ➔ ValidateAndImprove(t) //generate a prompt to enhance thought t,
- ➔ Aggregate(t1, ..., tk) //generate a prompt to combine thoughts t1, ..., tk
- ➔ Score(t) //score thought t
- ➔ Validate(t) //generate a prompt to validate the correctness of thought t

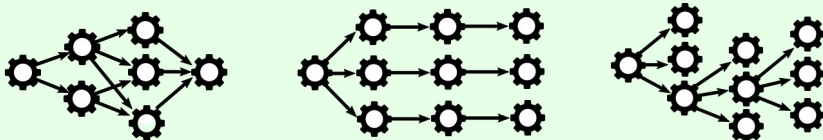
API for Parser (extensible)

ParseGenerate, ParseImprove, ParseScore,
ParseAggregate, ParseValidate, ...

//Each of the above routines is responsible for parsing an LLM thought
//to a corresponding Prompter routine (e.g., ParseScore parses Score).

Specifying the Structure of the Graph of Operations (GoO)

Graph of Operations enables seamless specification of not only
GoT, but also existing schemes such as CoT, CoT-SC, ToT



API for Controller

- LLM 및 파라미터 설정

Available operations when building the GoO

- 사용할 Operation을 원하는 대로 추가 코드로 확장가능

API for Prompter

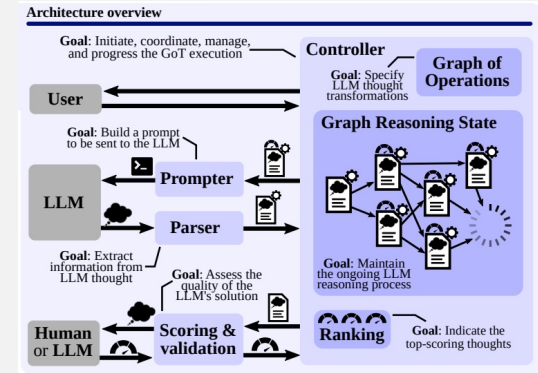
- 각 노드별 역할과 노드에서 활용할 프롬프트를 원하는 대로 확장가능

API for Parser

- LLM의 결과를 유의미하게 출력하도록 코드 정의

Specifying the Structure of the Graph of Operation (GoO)

- GoO를 정의할 때 원하는 구조를 설정하고 필요한 기능을 구현할 수 있음
- 구성하는 형태에 따라 CoT, CoT-SC, ToT 모두 구현 가능



* 참고용 이미지

System Architecture & Extensibility

Extensibility

Example of Graph Reasoning State Process

Example prompts and the Graph Reasoning State for the sorting use case

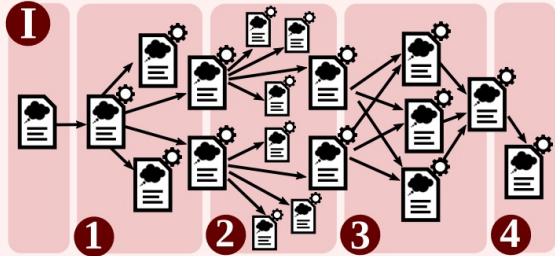
(sor

Initial/system prompt

(optional)

1

Hello. I want to sort the following input sequence of numbers: {input}



A prompt used by Generate(t, k=4)

1

<Instruction> Split the following list of 64 numbers into 4 lists of 16 numbers each, the first list should contain the first 16 numbers, the second list the second 16 numbers, the third list the third 16 numbers and the fourth list the fourth 16 numbers. Only output the final 4 lists in the following format without any additional text or thoughts!

```
{  
  "List 1": [3, 4, 3, 5, 7, 8, 1, ...],  
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...],  
  "List 3": [6, 9, 8, 1, 9, 2, 4, ...],  
  "List 4": [9, 0, 7, 6, 5, 6, 6, ...]  
}
```

<Example>

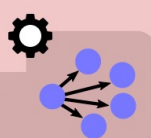
Input: [3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0, 9, 7, 2, 2, 4, 4, 8, 5, 0, 8, 7, 3, 3, 8, 7, 0, 9, 5, 1, 6, 7, 6, 8, 9, 0, 3, 0, 6, 3, 4, 8, 0, 6, 9, 8, 4, 1, 2, 9, 0, 4, 4, 8, 8, 9, 9, 8, 5, 9]
Output:

```
{  
  "List 1": [3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0],  
  "List 2": [9, 7, 2, 2, 4, 4, 8, 5, 0, 8, 7, 3, 3, 8, 7, 0],  
  "List 3": [9, 5, 1, 6, 7, 6, 8, 9, 0, 3, 0, 6, 3, 4, 8, 0],  
  "List 4": [6, 9, 8, 4, 1, 2, 9, 0, 4, 4, 8, 8, 9, 9, 8, 5, 9]  
}
```

<Example>

Input: {input}

The input thought t



This prompt is used by an operation Generate where the branching factor is $k=4$. Four new thoughts are constructed based on the LLM reply to this prompt.

A prompt used by Generate(t, k=1)+Repeat(k=4)

2

<Instruction> Sort the following list of numbers in ascending order. Output only the sorted list of numbers, no additional text. </Instruction>

<Example>

Input: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9, 4, 3, 5, 6, 6, 4, 4, 5, 2, 0, 9, 3, 3, 9, 2, 1]
Output: [0, 0, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9]
</Example>

Input: {input}

The input thought t

This prompt is used by an operation Generate where the branching factor is $k=1$, which means, only one thought is generated. However, as we chain it with the operation Repeat with $k=4$, the underlying GoT framework ensures that Generate executes 4 times and results in 4 separate thoughts. Note that, from the graph theory perspective, the GRS is identical to that in the operation Generate($t, k=4$). The difference between these two is that Generate($t, k=4$) gives the user more control over how these multiple thoughts are constructed, while Generate($t, k=1$)+Repeat($k=4$) is less flexible but more easy to use. Moreover, with Repeat one has 4 context-isolated responses from the LLM for identical prompts, whereas without Repeat there is only one context where all 4 thoughts are generated and must be explicitly handled in a single prompt/session.

A prompt used by

Aggregate(t1, t2)+Repeat(k=3)+KeepBest(N=1)

3

<Instruction> Merge the following 2 sorted lists of length {length1} each, into one sorted list of length {length2} using a merge sort style approach. Only output the final merged list without any additional text or thoughts! </Instruction>

<Approach>

To merge the two lists in a merge-sort style approach, follow these steps:
1. Compare the first element of both lists.
2. Append the smaller element to the merged list and move to the next element in the list from which the smaller element came.
3. Repeat steps 1 and 2 until one of the lists is empty.
4. Append the remaining elements of the non-empty list to the merged list. </Approach>

Merge the following two lists into one sorted list:

1: {input1}

2: {input2}

Merged list:

The input thoughts t1, t2

This prompt is used by an operation Aggregate where the aggregation factor is $k=2$ (2 input thoughts, $t1$ and $t2$, are aggregated). This is repeated by GoT 3 times, to maximize quality. Finally, the best result is selected. Note that, in this example, the prompt explicitly requests the merge operation only. All the remaining operations are specified in the GoO and are handled by the underlying GoT framework.

A prompt used by Improve(t)+Repeat(k=4)

4

<Instruction> The following two lists represent an unsorted list of numbers and a sorted variant of that list. The sorted variant is not correct. Fix the sorted variant so that it is correct. Make sure that the output list is sorted in ascending order, has the same number of elements as the input list ({length}), and contains the same elements as the input list. </Instruction>

<Approach>

To fix the incorrectly sorted list follow these steps:

1. For each number from 0 to 9, compare the frequency of that number in the incorrectly sorted list to the frequency of that number in the input list.
2. Iterate through the incorrectly sorted list and add or remove numbers as needed to make the frequency of each number in the incorrectly sorted list match the frequency of that number in the input list. </Approach>

<Examples>

Input: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9]

Incorrectly Sorted: [0, 0, 0, 0, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 7, 7, 8, 8, 9, 9, 9, 9]
Reason: The incorrectly sorted list contains four extra 0s, two extra 4s and three extra 9s and is missing two 2s.

Output: [0, 1, 2, 2, 2, 2, 3, 3, 4, 5, 5, 7, 7, 8, 8, 9]

Input: [6, 4, 5, 7, 5, 6, 9, 7, 6, 9, 4, 6, 9, 8, 1, 9, 2, 4, 9, 0, 7, 6, 5, 6, 6, 2, 8, 3, 9, 5, 6, 1]
Incorrectly Sorted: [0, 1, 1, 2, 2, 3, 4, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9, 9]
Reason: The incorrectly sorted list contains two extra 4s and is missing two 6s and one 9.

Output: [0, 1, 1, 2, 2, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9, 9, 9, 9]
</Examples>

Input: {input}

Incorrectly Sorted: {incorrectly_sorted}

The input thought t

This prompt is used by an operation Improve(t), which enhances a given thought t using information provided in another thought. Depending on how the Improve + Repeat operation is implemented by the user within GoT, it can either generate a number of new thoughts in GRS (the upper graph on the right), similar to Generate + Repeat, or may refine the same thought in GRS (the lower graph on the right), chaining $k=4$ refinement iterations together.

1

Length가 64인 리스트를
16개인 리스트 4개로 분할

2

Length가 16개인 리스트를 Sorting
(4 번 반복)

3

Length가 16개인 Sorted List 2개를 병합
(3번 반복), 가장 좋은 1개 선택

4

Sorting이 잘못된 리스트가 왜
잘못되었는지 예시를 활용하여 개선
(4번 반복)

Q & A

Example Use Cases

01 Sorting

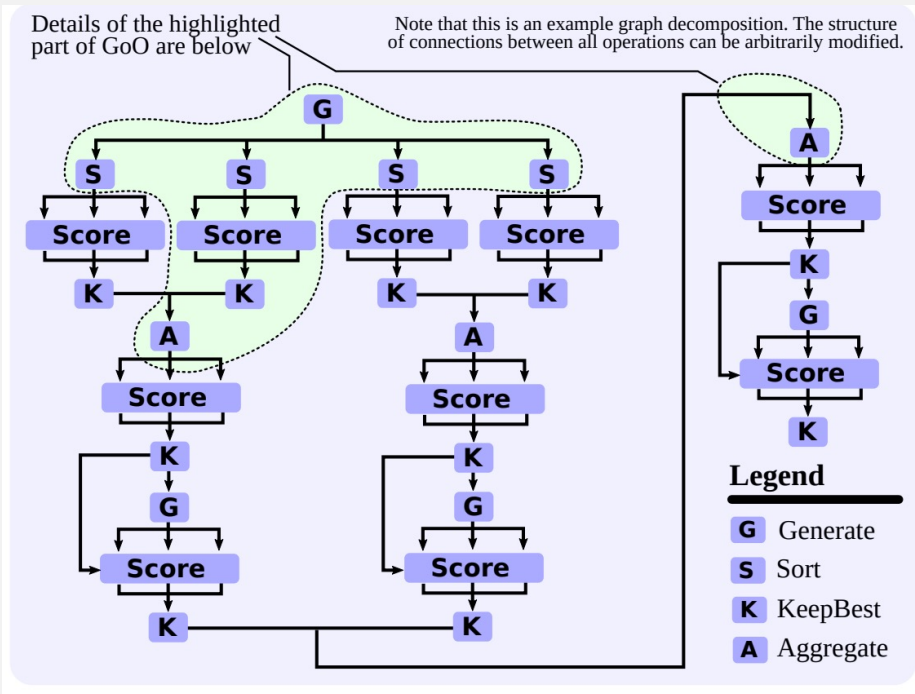
02 Set Operations

03 Keyword Counting

04 Document Merging

Example Use Cases

GoT에 적합한 Task 예시



- Sorting (정렬)

Table 5: Sorting of a 32 element list: Execution plan (**GoO**)

GoO:

1. Split the input list into two sub-lists of equal size (split_prompt)
2. For each sub-list: Sort the sub-list (sort_prompt) five times; score each sort attempt; keep the best
3. Merge the sorted sub-lists into one fully sorted list (merge_prompt) 10 times; score each merge attempt; keep the best
4. Fix any potential mistakes in the sorted list (improve_prompt) 10 times; score each improvement attempt; keep the best

List 를 작게 분할 > 각각을 정렬 > 긴 Sorted List로 통합 > 최종 결과 개선

- Set Operation (교집합연산)

Table 12: Intersection of two 32-element sets: Execution plan (GoO)

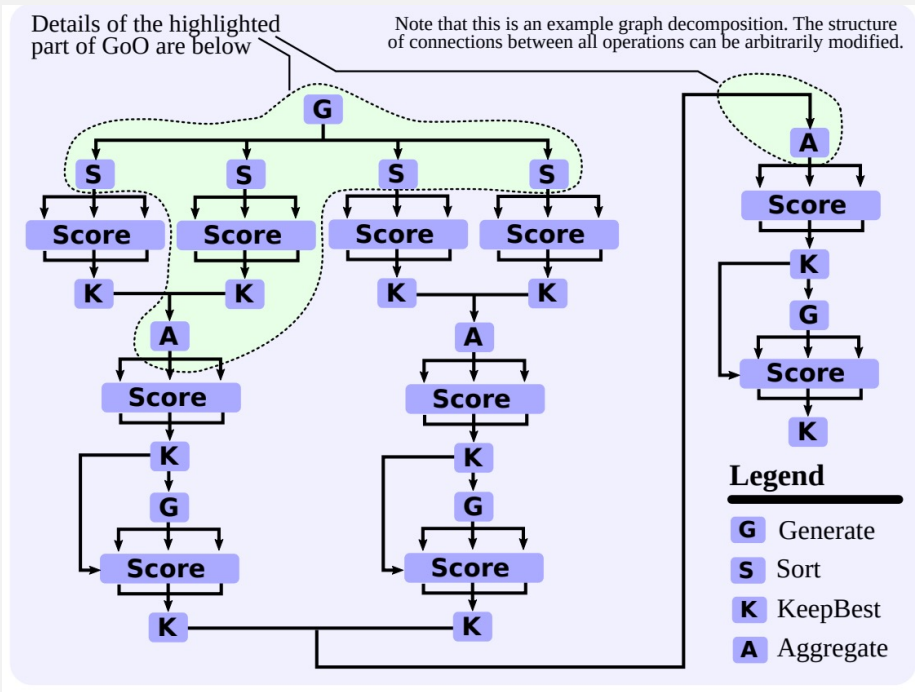
GoO:

1. Split the second input set into two sub-sets of equal size (split_prompt)
2. For each sub-set: Intersect the sub-set with the first input set (intersect_prompt) five times; score each sort attempt; keep the best
3. Merge the resulting intersections into one full intersection set (merge_prompt) 10 times; score each merge attempt; keep the best

Set을 작게 분할 > Set operation > 긴 Set으로 통합

Example Use Cases

GoT에 적합한 Task 예시



- Keyword Counting (문단내 키워드 개수 측정)

Table 20: Keyword counting for an example 4-passage split (GoT4): Execution plan (**GoO**)

GoO:

1. Split the input text into four paragraphs of roughly equal size (split_prompt)
2. For each paragraph: Count the occurrences of individual countries (count_prompt) 10 times; score each counting attempt; keep the best
3. Merge the country counts into one dictionary (merge_prompt) 3 times; validate and improve invalid merge attempts (improve_merge_prompt) up to 3 attempts each; score; keep the best

문단을 작게 분할 -> Keyword Counting -> 큰 문단으로 통합

- Document Merging (새로운 정보는 취합하고 겹치는 문장을 제거하는 task)

Table 34: Merging 4 example NDAs: Execution plan (GoO)

GoO:

1. Merge the 4 NDAs into a single one 5 times; Score each attempt and keep the best 3
2. Aggregate the merge attempts into a single one 5 times; Score each aggregation attempt and keep the overall best attempt (including Step 1)
3. Improve the merged NDA 10 times; Score each and keep the best

4개 문서를 하나로 병합 후 3개 추출 → 병합된 문단을 활용해 통합 → 결과 개선

The Latency-Volume Tradeoff

효율성을 평가하기 위한 새로운 Metric

Scheme	Latency	Volume
Chain-of-Thought (CoT)	N	N
Self-Consistency with CoT (CoT-SC)	N/k	N/k
Tree of Thoughts (ToT)	$\log_k N$	$O(\log_k N)$
Graph of Thoughts (GoT)	$\log_k N$	N

Table 2: Comparison of prompting schemes, with respect to their fundamental tradeoff between latency and volume. **GoT offers the best tradeoff.**

* $O(1)$: 하나의 thought를 생성하는데 필요한 비용

- Latency

정의 : 최종 결과를 도출하는데 걸리는 시간

- CoT는 Latency가 정답을 도출하기 위해 N 개의 Thought를 생성
- CoT-SC는 k 배 더 생기므로 느려짐
- ToT는 k 의 지수함수 형태로 생기므로 $\log$ 형태로 느려짐
- GoT도 각 생성이 지수함수 형태로 생기므로 $\log$ 형태로 느려짐

- Volume

정의 : 최종 결과를 도출하는데 사용한 thought 개수

- CoT는 최종 결과를 위해 N 을 모두 사용
- CoT-SC는 k 개 중 하나의 Reasoning path만 사용
- ToT는 지수만큼 늘어난 Reasoning path중 하나의 경로만 사용
- GoT는 병합하는 과정으로 인해 이상적으로 모든 thought를 사용가능

Evaluation

01 Evaluation Methodology

02 Analysis of GoT's Advantages

03 Discussion on Task Decomposition

Evaluation **Evaluation Methodology**

Evaluation Environment

- Parameter

Data samples : Task 별 100개의 데이터 구축

Temperature : 1.0

Context Size : 4k

* Fix number of thoughts

- Diversity of ToT architecture

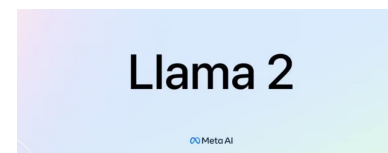
ToT : 많은 브랜치 생성 / 낮은 depth of tree

ToT2 : 적은 브랜치 생성 / 높은 depth of tree

- Used LLM



GPT-3.5



Llama 7b / 70b

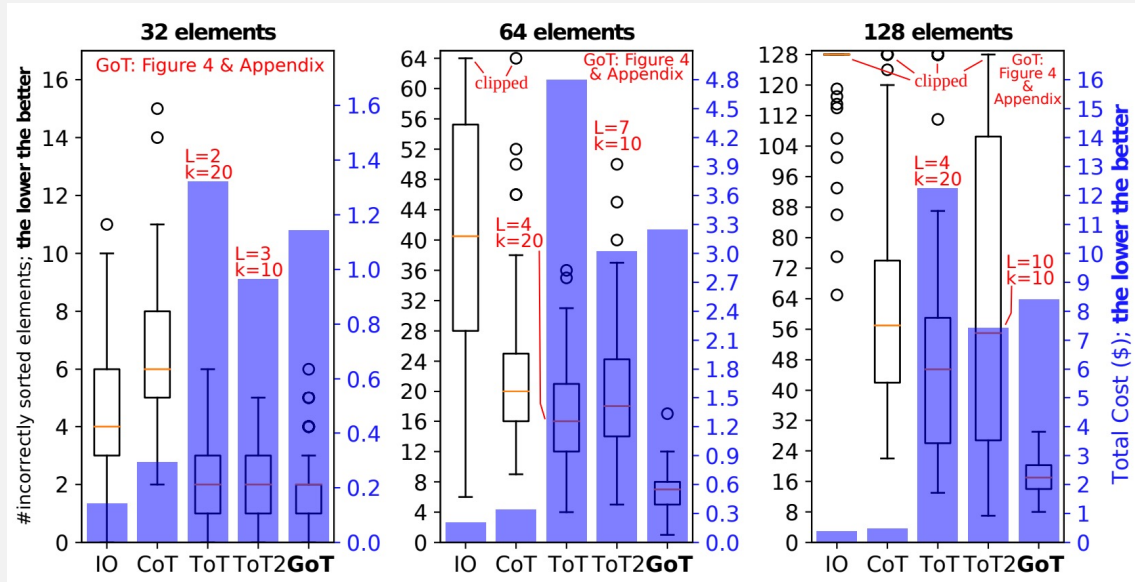
- 결과적으로 Llama2가 GPT-3.5에 비해 성능이 낮으며 느림
- 실험결과는 GPT-3.5을 집중하여 작성됨

Evaluation Analysis of GoT's Advantages

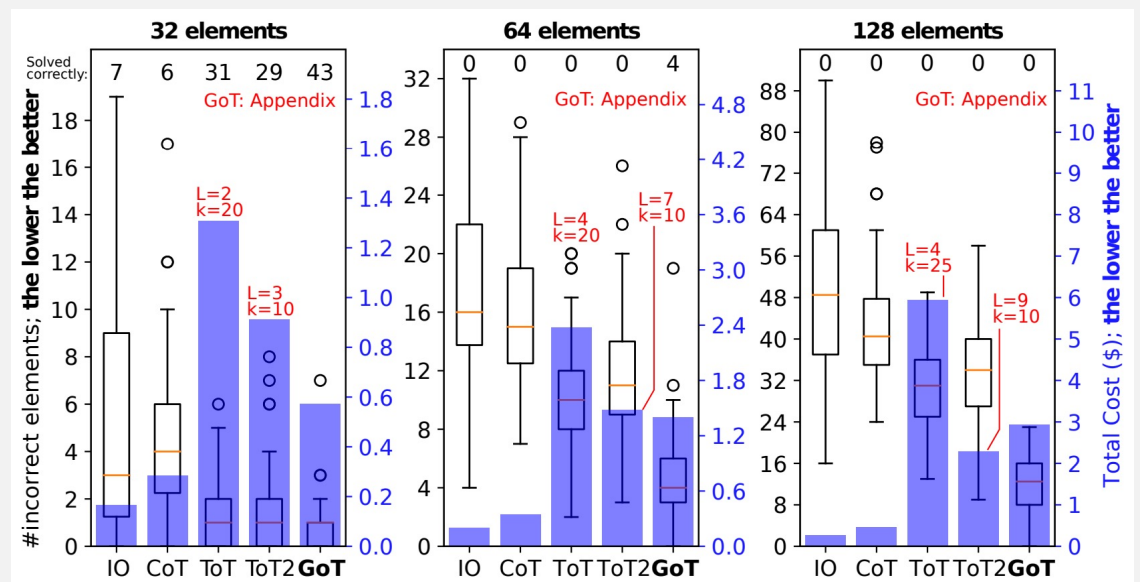
Advantages of GoT

- GoT vs ToT
 - Cost가 적고 Error가 적음
- GoT vs IO and CoT
 - Quality가 IO, CoT에 비해 크게 향상
- Increasing Complexity of Tackled Problems
 - 복잡도가 늘어날수록 GoT의 이점이 검증됨

Sorting



Set Operation

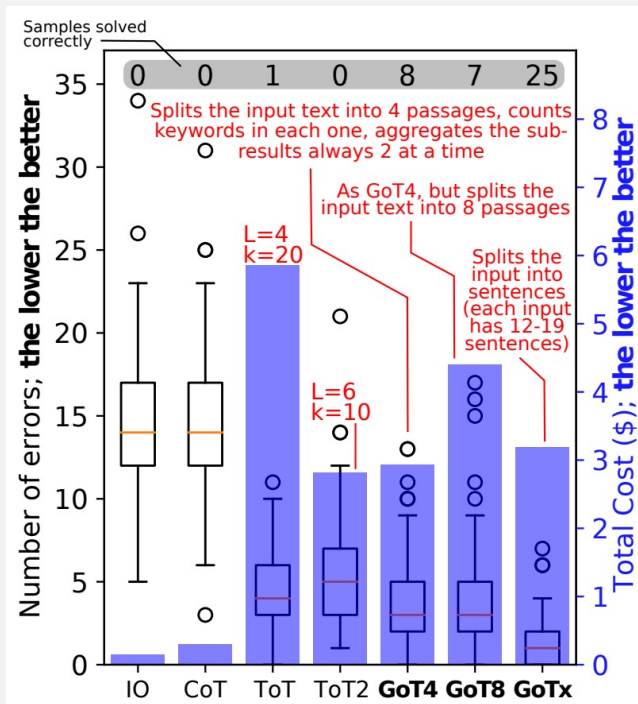


Evaluation Analysis of GoT's Advantages

Advantages of GoT

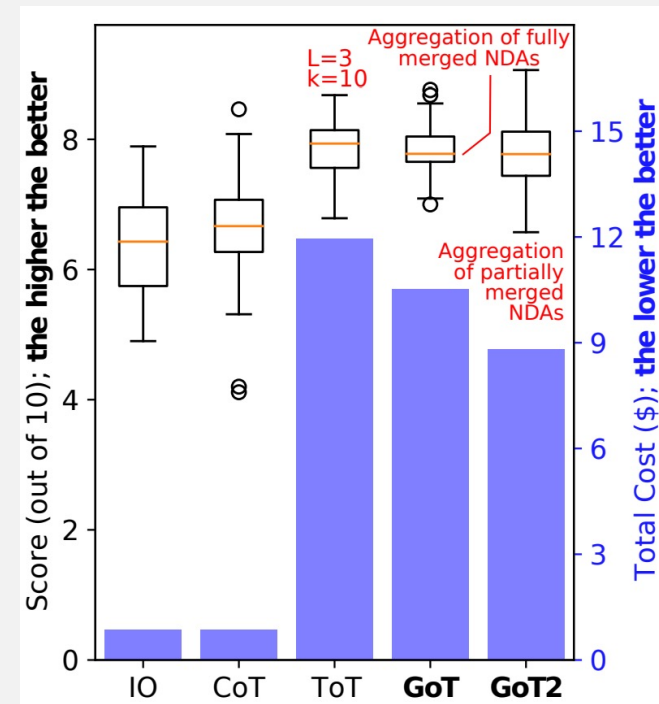
- GoT vs ToT
 - Cost가 적고 Error가 적음
- GoT vs IO and CoT
 - Quality가 IO, CoT에 비해 크게 향상
- Increasing Complexity of Tackled Problems
 - 복잡도가 늘어날수록 GoT의 이점이 검증됨

Keyword counting



- GoT4
문단을 4개로 분리
- GoT8
문단을 8개로 분리
- GoTx
문단을 12~19개로 분리

Document Merging



- GoT
병합 문서를 바로 aggregation transform 적용
- GoT2
병합 문서를 부분적으로 aggregation transform 적용

Evaluation Discussion on Task Decomposition

Task Decomposition

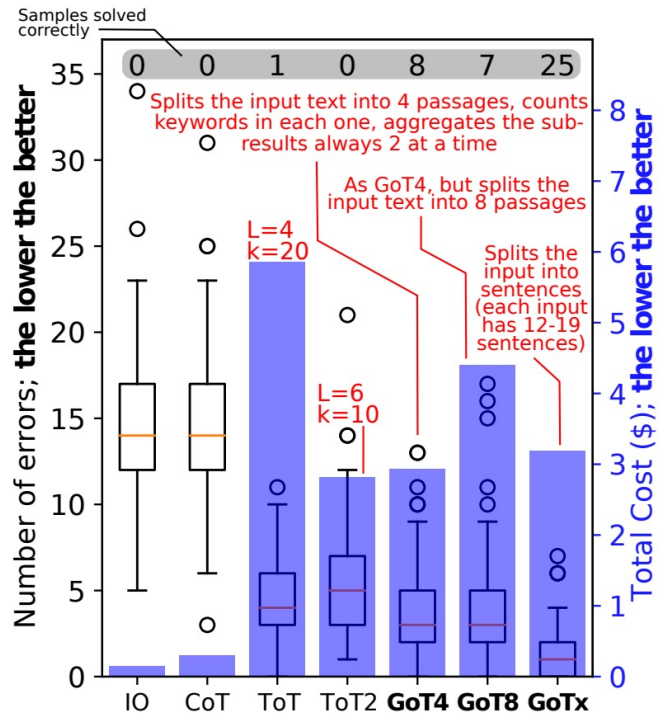


Figure 7: Number of errors and cost in keyword counting with ChatGPT-3.5. L and k indicate the structure of ToT (see Sections 3.2 and 6).

- Cost

- 분할을 많이 할수록 비용이 계속해서 커지는 아키텍처가 아님
- 분할을 할수록 문장단위가 짧아지기도 해서 비용이 적어질 수 있음

- Performance

- 큰 문제를 하나의 프롬프트로 해결가능한 점 단위의 문제로 쪼개야 함
- 앞서 문제를 쉽게 해결했다면 improvement / refinement cost를 줄일 수 있음
- 저자의 실험에서는 바닥부터 단계적으로 해결하는 것보다 작게 쪼갠 결과를 합치는 방향이 LLM에게 쉬운 것이 확인됨

Conclusion

- **Propose Graph of Thoughts(GoT), a new paradigm that enable the LLM to solve different tasks**
- **GoT outperforms other prompting schemes. 62% increase in the quality of sorting over ToT. while simultaneously reducing costs by > 31%**
- **Propose a novel metric for a prompting scheme, the volume of a thought, to indicate the scope of information that a given LLM output could carry with it**

Q & A

